File: backend_fastapi_mastery/01_fastapi_core_concepts.md

FastAPI Core Concepts: Deep Mastery

Why FastAPI?

FastAPI isn't just another Python web framework. It's built on modern Python features (type hints, async/await) and designed for building production APIs. Understanding *why* FastAPI makes certain design decisions will help you use it effectively and explain your choices in interviews.

Key design principles:

- **Type hints drive everything**: Validation, serialization, documentation
- **Async-first but sync-friendly**: Use async when beneficial, sync when simpler
- **Dependency injection as core pattern**: Not an afterthought
- **Standards-based**: OpenAPI, JSON Schema, OAuth2

Application Instantiation

Basic App Creation

```
from fastapi import FastAPI

app = FastAPI(
    title="Payment Service",
    description="Handles payment processing and reconciliation",
    version="1.0.0",
    docs_url="/docs",        # Swagger UI
    redoc_url="/redoc",      # ReDoc
    openapi_url="/openapi.json"
)
```

Production Configuration

```
from fastapi import FastAPI
from contextlib import asynccontextmanager

# Modern approach: lifespan context manager (FastAPI 0.95+)
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: runs before accepting requests
```

```
    print("Starting up...")
    await database.connect()
    await cache.connect()

    yield  # Application runs here

    # Shutdown: runs after last request
    print("Shutting down...")
    await database.disconnect()
    await cache.disconnect()

app = FastAPI(
    title="Payment Service",
    version="1.0.0",
    lifespan=lifespan,
    # Production: often disable docs
    docs_url=None if settings.ENVIRONMENT == "production" else "/docs",
    redoc_url=None if settings.ENVIRONMENT == "production" else "/redoc",
)
```

Why Lifespan Matters

Common mistake: Initializing database connections at module load time.

```
# WRONG: Connection created at import time
# If import fails or runs in wrong context, disaster
db = Database(settings.DATABASE_URL)
db.connect()  # Blocks, might fail

# RIGHT: Use lifespan for controlled initialization
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.db = Database(settings.DATABASE_URL)
    await app.state.db.connect()
    yield
    await app.state.db.disconnect()
```

Production insight: In Kubernetes/containerized environments, the startup phase is when health checks begin. If your app crashes during startup, the orchestrator restarts it. Clean lifespan management means predictable startup behavior.

Path Operations (Routes)

The Decorator Pattern

Every endpoint in FastAPI is a **path operation** - a combination of:

- HTTP method (GET, POST, etc.)
- Path (/users, /orders/{id})

- Function that handles the request

```
@app.get("/users/{user_id}")
async def get_user(user_id: int):
    """
    Retrieve a user by ID.

    - **user_id**: The unique identifier of the user
    """
    return {"user_id": user_id}
```

Path Parameters

```
from fastapi import Path

@app.get("/users/{user_id}")
async def get_user(
    user_id: int = Path(
        ...,  # Required (no default)
        title="User ID",
        description="The unique identifier of the user",
        ge=1,  # Greater than or equal to 1
        example=12345
    )
):
    return await user_repository.find(user_id)
```

Path parameter validation happens automatically:

- Type conversion (string "123" → int 123)
- Constraint validation (ge=1 checks value >= 1)
- Returns 422 on validation failure

Query Parameters

```
from fastapi import Query
from typing import Optional, List

@app.get("/users")
async def list_users(
    # Required query param
    status: str = Query(..., description="Filter by status"),

    # Optional with default
    limit: int = Query(default=20, ge=1, le=100),
    offset: int = Query(default=0, ge=0),

    # Optional, can be None
```

```
    search: Optional[str] = Query(default=None, min_length=1),

    # List of values: /users?tag=python&tag=fastapi
    tags: List[str] = Query(default=[]),
):
    return await user_repository.list(
        status=status,
        limit=limit,
        offset=offset,
        search=search,
        tags=tags
    )
```

Request Body

```
from pydantic import BaseModel, Field
from typing import Optional

class UserCreate(BaseModel):
    email: str = Field(..., example="user@example.com")
    name: str = Field(..., min_length=1, max_length=100)
    age: Optional[int] = Field(default=None, ge=0, le=150)

    class Config:
        json_schema_extra = {
            "example": {
                "email": "john@example.com",
                "name": "John Doe",
                "age": 30
            }
        }

@app.post("/users", status_code=201)
async def create_user(user: UserCreate):
    """
    Create a new user.

    The request body is automatically validated against UserCreate schema.
    """
    return await user_repository.create(user)
```

Multiple Input Sources

```
@app.put("/users/{user_id}")
async def update_user(
    # From path
    user_id: int = Path(..., ge=1),

    # From query string
```

```
    notify: bool = Query(default=True),

    # From request body
    user: UserUpdate = ...,

    # From headers
    x_request_id: Optional[str] = Header(default=None),

    # From cookies
    session_id: Optional[str] = Cookie(default=None),
):
    # All parameters validated and converted automatically
    pass
```

Response Configuration

```
from fastapi import status
from fastapi.responses import JSONResponse

class UserResponse(BaseModel):
    id: int
    email: str
    name: str
    created_at: datetime

@app.post(
    "/users",
    response_model=UserResponse,
    status_code=status.HTTP_201_CREATED,
    responses={
        201: {"description": "User created successfully"},
        409: {"description": "User with this email already exists"},
        422: {"description": "Validation error"},
    },
    tags=["users"],
    summary="Create a new user",
)
async def create_user(user: UserCreate):
    # Even if you return extra fields, response_model filters them
    db_user = await user_repository.create(user)
    return db_user  # Only fields in UserResponse are returned
```

Production insight: `response_model` is crucial for security. It prevents accidentally leaking sensitive fields like `password_hash` that exist in your database model but shouldn't be in API responses.

Dependency Injection System

This is FastAPI's most powerful feature. Understanding it deeply separates beginners from advanced users.

What is Dependency Injection?

Instead of creating dependencies inside your functions, you **declare** them and let the framework provide them.

```
# WITHOUT dependency injection
@app.get("/users/{user_id}")
async def get_user(user_id: int):
    db = Database()  # Created every request - WRONG
    user = db.query(User).filter_by(id=user_id).first()
    db.close()
    return user

# WITH dependency injection
async def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/users/{user_id}")
async def get_user(user_id: int, db: Session = Depends(get_db)):
    return db.query(User).filter_by(id=user_id).first()
```

Dependency Types

1. Simple callable dependencies:

```
from fastapi import Depends

def get_settings():
    return Settings()

@app.get("/config")
async def get_config(settings: Settings = Depends(get_settings)):
    return {"debug": settings.debug}
```

2. Generator dependencies (with cleanup):

```
async def get_db():
    db = SessionLocal()
    try:
        yield db  # Value provided to route
    finally:
        db.close()  # Cleanup after request

@app.get("/users")
```

```
async def list_users(db: Session = Depends(get_db)):
    # db is available here
    return db.query(User).all()
    # After response, finally block runs
```

3. Class-based dependencies:

```
class Pagination:
    def __init__(
        self,
        limit: int = Query(default=20, ge=1, le=100),
        offset: int = Query(default=0, ge=0),
    ):
        self.limit = limit
        self.offset = offset

@app.get("/users")
async def list_users(pagination: Pagination = Depends()):
    # Depends() without argument uses the class itself
    return await user_repository.list(
        limit=pagination.limit,
        offset=pagination.offset
    )
```

4. Nested dependencies:

```
async def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

async def get_user_repository(db: Session = Depends(get_db)):
    return UserRepository(db)

async def get_current_user(
    token: str = Depends(oauth2_scheme),
    user_repo: UserRepository = Depends(get_user_repository)
):
    user_id = decode_token(token)
    user = await user_repo.find(user_id)
    if not user:
        raise HTTPException(status_code=401)
    return user

@app.get("/me")
async def get_me(current_user: User = Depends(get_current_user)):
    # Dependency chain: oauth2_scheme → get_db → get_user_repository →
```

```
get_current_user
    return current_user
```

Dependency Scopes

```
# Request-scoped (default): New instance per request
@app.get("/users")
async def list_users(db: Session = Depends(get_db)):
    pass  # Fresh db session for this request

# App-scoped: Share across all requests (use with caution)
# Store in app.state during lifespan
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.http_client = httpx.AsyncClient()
    yield
    await app.state.http_client.aclose()

def get_http_client(request: Request):
    return request.app.state.http_client

@app.get("/external")
async def call_external(client: httpx.AsyncClient =
Depends(get_http_client)):
    return await client.get("https://api.example.com")
```

Common Dependency Patterns

Authentication dependency:

```
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/token")

async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db)
) -> User:
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        user_id = payload.get("sub")
    except JWTError:
        raise HTTPException(
            status_code=401,
            detail="Invalid token",
            headers={"WWW-Authenticate": "Bearer"}
        )

    user = db.query(User).filter_by(id=user_id).first()
```

```
    if not user:
        raise HTTPException(status_code=401, detail="User not found")

    return user

async def get_current_active_user(
    current_user: User = Depends(get_current_user)
) -> User:
    if not current_user.is_active:
        raise HTTPException(status_code=403, detail="Inactive user")
    return current_user

async def get_admin_user(
    current_user: User = Depends(get_current_active_user)
) -> User:
    if not current_user.is_admin:
        raise HTTPException(status_code=403, detail="Admin access
required")
    return current_user
```

Rate limiting dependency:

```
from fastapi import Request
import time

class RateLimiter:
    def __init__(self, requests_per_minute: int = 60):
        self.requests_per_minute = requests_per_minute
        self.requests = {}  # In production, use Redis

    async def __call__(self, request: Request):
        client_ip = request.client.host
        current_time = time.time()

        # Clean old entries
        self.requests = {
            ip: times for ip, times in self.requests.items()
            if any(t > current_time - 60 for t in times)
        }

        # Check rate
        client_requests = self.requests.get(client_ip, [])
        recent_requests = [t for t in client_requests if t > current_time
- 60]

        if len(recent_requests) >= self.requests_per_minute:
            raise HTTPException(
                status_code=429,
                detail="Rate limit exceeded",
                headers={"Retry-After": "60"}
            )
```

```
        self.requests[client_ip] = recent_requests + [current_time]

rate_limiter = RateLimiter(requests_per_minute=60)

@app.get("/api/data")
async def get_data(_: None = Depends(rate_limiter)):
    return {"data": "..."}
```

Request and Response Models

Request Validation Flow

When a request arrives:

1. **Path parameters extracted and validated**
2. **Query parameters extracted and validated**
3. **Headers extracted and validated**
4. **Body parsed as JSON**
5. **Body validated against Pydantic model**
6. **If any validation fails → 422 response**
7. **Dependencies resolved (may do additional validation)**
8. **Route function called**

```
from pydantic import BaseModel, Field, validator
from typing import Optional
from datetime import datetime

class OrderCreate(BaseModel):
    product_id: int = Field(..., gt=0)
    quantity: int = Field(..., ge=1, le=100)
    notes: Optional[str] = Field(default=None, max_length=500)

    @validator("quantity")
    def validate_quantity(cls, v, values):
        # Custom validation logic
        return v

@app.post("/orders")
async def create_order(order: OrderCreate):
    # If we reach here, order is fully validated
    pass
```

Response Model Filtering

```
class UserInDB(BaseModel):
    id: int
    email: str
```

```
    name: str
    password_hash: str  # Sensitive!
    created_at: datetime

class UserResponse(BaseModel):
    id: int
    email: str
    name: str
    created_at: datetime

@app.get("/users/{user_id}", response_model=UserResponse)
async def get_user(user_id: int, db: Session = Depends(get_db)):
    user = db.query(UserModel).filter_by(id=user_id).first()
    # Even though user has password_hash, it's filtered out
    return user
```

Response Model Options

```
@app.get(
    "/users/{user_id}",
    response_model=UserResponse,
    response_model_exclude_unset=True,  # Omit fields not explicitly set
    response_model_exclude_none=True,   # Omit None values
    response_model_by_alias=True,       # Use field aliases
)
async def get_user(user_id: int):
    pass
```

HTTPException and Error Handling

Basic Exception Handling

```
from fastapi import HTTPException, status

@app.get("/users/{user_id}")
async def get_user(user_id: int, db: Session = Depends(get_db)):
    user = db.query(User).filter_by(id=user_id).first()
    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="User not found"
        )
    return user
```

Custom Exception Classes

```python
from fastapi import HTTPException

class NotFoundError(HTTPException):
    def __init__(self, resource: str, resource_id: any):
        super().__init__(
            status_code=404,
            detail=f"{resource} with id {resource_id} not found"
        )

class ConflictError(HTTPException):
    def __init__(self, detail: str):
        super().__init__(status_code=409, detail=detail)

class UnauthorizedError(HTTPException):
    def __init__(self, detail: str = "Invalid credentials"):
        super().__init__(
            status_code=401,
            detail=detail,
            headers={"WWW-Authenticate": "Bearer"}
        )

# Usage
@app.get("/users/{user_id}")
async def get_user(user_id: int):
    user = await user_repo.find(user_id)
    if not user:
        raise NotFoundError("User", user_id)
    return user
```

Global Exception Handlers

```python
from fastapi import Request
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError
from starlette.exceptions import HTTPException as StarletteHTTPException

@app.exception_handler(StarletteHTTPException)
async def http_exception_handler(request: Request, exc:
StarletteHTTPException):
    return JSONResponse(
        status_code=exc.status_code,
        content={
            "error": {
                "code": f"HTTP_{exc.status_code}",
                "message": exc.detail,
            },
            "request_id": getattr(request.state, "request_id", None)
        }
    )
```

```
@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc:
RequestValidationError):
    return JSONResponse(
        status_code=422,
        content={
            "error": {
                "code": "VALIDATION_ERROR",
                "message": "Invalid request data",
                "details": exc.errors()
            }
        }
    )

@app.exception_handler(Exception)
async def unhandled_exception_handler(request: Request, exc: Exception):
    # Log the full exception
    logger.exception(f"Unhandled exception: {exc}")

    # Return generic error to client (don't leak internal details)
    return JSONResponse(
        status_code=500,
        content={
            "error": {
                "code": "INTERNAL_ERROR",
                "message": "An unexpected error occurred"
            }
        }
    )
```

Lifecycle Events

Modern Lifespan Pattern (Recommended)

```
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    print("Connecting to database...")
    await database.connect()

    print("Warming up caches...")
    await cache.warmup()

    print("Starting background tasks...")
    background_scheduler.start()

    yield  # Application serves requests
```

```
    # Shutdown
    print("Stopping background tasks...")
    background_scheduler.stop()

    print("Disconnecting from database...")
    await database.disconnect()

    print("Cleanup complete")

app = FastAPI(lifespan=lifespan)
```

Legacy Event Handlers (Deprecated but may see in older code)

```
# Don't use in new code, but recognize it
@app.on_event("startup")
async def startup():
    await database.connect()

@app.on_event("shutdown")
async def shutdown():
    await database.disconnect()
```

Production Lifespan Patterns

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Initialize in specific order
    logger.info("Starting application...")

    # 1. Configuration
    config = load_config()
    app.state.config = config

    # 2. Database connections
    db_pool = await asyncpg.create_pool(config.database_url)
    app.state.db_pool = db_pool

    # 3. Cache connections
    redis = await aioredis.from_url(config.redis_url)
    app.state.redis = redis

    # 4. External service clients
    http_client = httpx.AsyncClient(timeout=30.0)
    app.state.http_client = http_client

    # 5. Health check ready
    app.state.ready = True
    logger.info("Application ready")
```

```
    yield

    # Shutdown in reverse order
    app.state.ready = False
    logger.info("Shutting down...")

    await http_client.aclose()
    await redis.close()
    await db_pool.close()

    logger.info("Shutdown complete")
```

Async vs Sync in FastAPI

The Mental Model

FastAPI runs on an **async event loop**. Understanding when to use async vs sync is critical for performance.

```
┌──────────────────────────────────────────────────────────┐
│                    Event Loop (uvicorn)                  │
│                                                          │
│    ┌─────────┐    ┌─────────┐    ┌─────────┐            │
│    │ Request │    │ Request │    │ Request │    ...     │
│    │    1    │    │    2    │    │    3    │            │
│    └────┬────┘    └────┬────┘    └────┬────┘            │
│         │              │              │                 │
│         ▼              ▼              ▼                 │
│    ┌──────────────────────────────────────────┐         │
│    │         async def route_handler()        │         │
│    │                                          │         │
│    │   await db.query()  ← Non-blocking!      │         │
│    │   await http.get()  ← Non-blocking!      │         │
│    │                                          │         │
│    │   While waiting, event loop handles      │         │
│    │   other requests!                        │         │
│    └──────────────────────────────────────────┘         │
└──────────────────────────────────────────────────────────┘
```

When to Use Async

```
# USE ASYNC: I/O bound operations with async libraries
import httpx
import asyncpg

@app.get("/data")
async def get_data():
    # These await calls release the event loop
```

```
    async with httpx.AsyncClient() as client:
        external_data = await client.get("https://api.example.com")

    # Async database query
    row = await database.fetch_one("SELECT * FROM users WHERE id = $1",
user_id)

    return {"external": external_data.json(), "db": row}
```

When to Use Sync

```
# USE SYNC: CPU-bound work or blocking libraries
import time
import some_blocking_library

@app.get("/compute")
def compute_heavy():  # Note: def, not async def
    """
    FastAPI automatically runs sync functions in a thread pool.
    This prevents blocking the event loop.
    """
    result = some_blocking_library.calculate()  # Blocking call
    return {"result": result}
```

The Critical Mistake

```
# WRONG: Blocking call in async function
@app.get("/data")
async def get_data():
    # requests is synchronous - BLOCKS THE ENTIRE EVENT LOOP
    response = requests.get("https://api.example.com")  # BAD!
    return response.json()

# RIGHT: Use async library
@app.get("/data")
async def get_data():
    async with httpx.AsyncClient() as client:
        response = await client.get("https://api.example.com")
    return response.json()

# ALSO RIGHT: Use sync function (runs in thread pool)
@app.get("/data")
def get_data():
    response = requests.get("https://api.example.com")
    return response.json()
```

Performance Implications

```
# Scenario: 100 concurrent requests, each calling external API (100ms
latency)

# ASYNC APPROACH
@app.get("/async-data")
async def async_data():
    await asyncio.sleep(0.1)  # Simulates 100ms I/O
    return {"data": "..."}
# Result: ~100ms total (all requests handled concurrently)

# SYNC APPROACH
@app.get("/sync-data")
def sync_data():
    time.sleep(0.1)  # Simulates 100ms I/O
    return {"data": "..."}
# Result: ~100ms * (100 / thread_pool_size)
# With default 40 threads: ~250ms

# WRONG: BLOCKING IN ASYNC
@app.get("/broken-data")
async def broken_data():
    time.sleep(0.1)  # BLOCKS EVENT LOOP
    return {"data": "..."}
# Result: ~10 seconds (requests processed sequentially!)
```

ASGI vs WSGI

WSGI (Web Server Gateway Interface)

Traditional Python web standard. **Synchronous only**.

```
# WSGI application signature
def application(environ, start_response):
    status = '200 OK'
    headers = [('Content-Type', 'text/plain')]
    start_response(status, headers)
    return [b'Hello World']

# Frameworks: Flask, Django (traditionally)
# Servers: Gunicorn, uWSGI
```

ASGI (Asynchronous Server Gateway Interface)

Modern standard. **Supports async and sync**.

```
# ASGI application signature
async def application(scope, receive, send):
```

```
    await send({
        'type': 'http.response.start',
        'status': 200,
        'headers': [[b'content-type', b'text/plain']],
    })
    await send({
        'type': 'http.response.body',
        'body': b'Hello World',
    })

# Frameworks: FastAPI, Starlette, Django 3+
# Servers: Uvicorn, Hypercorn, Daphne
```

Why ASGI Matters

Feature	WSGI	ASGI
Async support	No	Yes
WebSockets	No	Yes
HTTP/2	Limited	Yes
Long-polling	Inefficient	Efficient
Server-Sent Events	Hacky	Native

Running FastAPI

```
# Development
uvicorn main:app --reload

# Production (single process)
uvicorn main:app --host 0.0.0.0 --port 8000

# Production (multiple workers via gunicorn)
gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker --bind
0.0.0.0:8000
```

Router Organization

Why Routers?

As your API grows, you need to organize routes logically.

```
# main.py
from fastapi import FastAPI
from routers import users, orders, products
```

```
app = FastAPI(title="E-Commerce API")

app.include_router(users.router, prefix="/users", tags=["users"])
app.include_router(orders.router, prefix="/orders", tags=["orders"])
app.include_router(products.router, prefix="/products", tags=["products"])
```

```
# routers/users.py
from fastapi import APIRouter, Depends
from dependencies import get_db, get_current_user

router = APIRouter()

@router.get("/")
async def list_users(db = Depends(get_db)):
    return await db.query(User).all()

@router.get("/me")
async def get_me(current_user = Depends(get_current_user)):
    return current_user

@router.get("/{user_id}")
async def get_user(user_id: int, db = Depends(get_db)):
    return await db.query(User).filter_by(id=user_id).first()
```

Router-Level Dependencies

```
# All routes in this router require authentication
router = APIRouter(
    dependencies=[Depends(get_current_user)]
)

# All routes in this router use these response codes
router = APIRouter(
    responses={
        401: {"description": "Not authenticated"},
        403: {"description": "Not authorized"},
    }
)
```

Project Structure

```
my_api/
├── main.py              # App instantiation, router includes
├── config.py            # Settings, environment variables
├── dependencies.py      # Shared dependencies
├── database.py          # Database connection
├── models/              # SQLAlchemy/ORM models
```

```
│   ├── __init__.py
│   ├── user.py
│   └── order.py
├── schemas/            # Pydantic models
│   ├── __init__.py
│   ├── user.py
│   └── order.py
├── routers/            # Route handlers
│   ├── __init__.py
│   ├── users.py
│   └── orders.py
├── services/           # Business logic
│   ├── __init__.py
│   ├── user_service.py
│   └── order_service.py
├── repositories/       # Data access
│   ├── __init__.py
│   ├── user_repository.py
│   └── order_repository.py
└── tests/
    ├── __init__.py
    ├── conftest.py
    └── test_users.py
```

Anti-Patterns to Avoid

1. Business Logic in Route Handlers

```python
# WRONG: Route handler does everything
@app.post("/orders")
async def create_order(order: OrderCreate, db = Depends(get_db)):
    # Validation
    user = db.query(User).filter_by(id=order.user_id).first()
    if not user:
        raise HTTPException(404)

    # Business logic
    total = 0
    for item in order.items:
        product = db.query(Product).filter_by(id=item.product_id).first()
        if product.stock < item.quantity:
            raise HTTPException(400, "Insufficient stock")
        total += product.price * item.quantity
        product.stock -= item.quantity

    # More business logic
    if user.has_discount:
        total *= 0.9

    # Persistence
```

```
    db_order = Order(user_id=user.id, total=total)
    db.add(db_order)
    db.commit()

    # Side effects
    send_email(user.email, "Order confirmed")

    return db_order

# RIGHT: Route handler is thin
@app.post("/orders")
async def create_order(
    order: OrderCreate,
    order_service: OrderService = Depends(get_order_service)
):
    return await order_service.create_order(order)
```

2. Not Using Dependency Injection

```
# WRONG: Creating dependencies inline
@app.get("/users/{user_id}")
async def get_user(user_id: int):
    db = SessionLocal()  # New connection every request
    try:
        user = db.query(User).filter_by(id=user_id).first()
        return user
    finally:
        db.close()

# RIGHT: Dependency injection
@app.get("/users/{user_id}")
async def get_user(user_id: int, db = Depends(get_db)):
    return db.query(User).filter_by(id=user_id).first()
```

3. Blocking in Async Functions

```
# WRONG: Blocks event loop
@app.get("/data")
async def get_data():
    data = requests.get("https://api.example.com")  # BLOCKING!
    return data.json()

# RIGHT: Use async library or sync function
@app.get("/data")
async def get_data():
    async with httpx.AsyncClient() as client:
        response = await client.get("https://api.example.com")
    return response.json()
```

4. Missing Response Models

```
# WRONG: No response model, might leak sensitive data
@app.get("/users/{user_id}")
async def get_user(user_id: int, db = Depends(get_db)):
    return db.query(User).filter_by(id=user_id).first()
    # If User model has password_hash, it gets returned!

# RIGHT: Explicit response model
@app.get("/users/{user_id}", response_model=UserResponse)
async def get_user(user_id: int, db = Depends(get_db)):
    return db.query(User).filter_by(id=user_id).first()
```

Mastery Checkpoints

Conceptual Questions

1. **What happens when you declare a sync function (`def`) as a route handler in FastAPI?**

   Answer: FastAPI detects it's synchronous and runs it in a thread pool to prevent blocking the async event loop. This is why you can use blocking libraries in sync handlers without freezing your app. The default thread pool size is 40 threads.

2. **Why does FastAPI use Pydantic models for request bodies instead of just accepting dictionaries?**

   Answer: Pydantic provides: (1) Automatic validation with clear error messages, (2) Type conversion (string "123" to int), (3) IDE autocomplete and type checking, (4) Automatic API documentation generation, (5) Serialization control. Using dicts means manual validation everywhere.

3. **Explain the difference between `Depends(get_db)` and `get_db()`**

   Answer: `Depends(get_db)` tells FastAPI to manage the dependency lifecycle - it will call `get_db()`, handle the `yield`, and ensure cleanup runs after the request. `get_db()` directly calls the function, bypassing FastAPI's dependency injection, which means no lifecycle management and potential resource leaks.

4. **Why use the lifespan context manager instead of on_event decorators?**

   Answer: Lifespan is the modern pattern that (1) ensures cleanup runs even if startup fails partially, (2) provides a single place for all initialization/cleanup, (3) supports dependency sharing via yield, (4) is not deprecated. on_event decorators can leave resources dangling if startup fails between events.

5. **When would you use `status_code=202` instead of `201`?**

   Answer: 202 Accepted means the request is valid and accepted for processing, but processing will happen asynchronously. Use it when the resource won't exist immediately after the response (batch jobs, video processing, report generation). 201 Created means the resource exists now at the returned location.

Scenario Questions

6. **Design dependencies for a multi-tenant SaaS application where each request must be scoped to a tenant.**

   Answer:

```
async def get_tenant(
    x_tenant_id: str = Header(...),
    db: Session = Depends(get_db)
) -> Tenant:
    tenant = await db.query(Tenant).filter_by(id=x_tenant_id).first()
    if not tenant:
        raise HTTPException(404, "Tenant not found")
    return tenant

async def get_tenant_db(tenant: Tenant = Depends(get_tenant)):
    # Return database session scoped to tenant schema/connection
    return await get_tenant_scoped_session(tenant.database_url)

@app.get("/users")
async def list_users(db = Depends(get_tenant_db)):
    # Automatically scoped to tenant
    return await db.query(User).all()
```

7. **Your route handler needs to call three external APIs. How do you structure this efficiently?**

   Answer:

```
@app.get("/aggregated")
async def get_aggregated():
    async with httpx.AsyncClient() as client:
        # Run all three calls concurrently
        results = await asyncio.gather(
            client.get("https://api1.example.com"),
            client.get("https://api2.example.com"),
            client.get("https://api3.example.com"),
            return_exceptions=True  # Don't fail all if one fails
        )

    # Process results, handle any exceptions
    return {
        "api1": results[0].json() if not isinstance(results[0],
Exception) else None,
        "api2": results[1].json() if not isinstance(results[1],
Exception) else None,
        "api3": results[2].json() if not isinstance(results[2],
Exception) else None,
    }
```

8. **How would you implement request validation that depends on database state?**

   Answer:

```
class OrderCreate(BaseModel):
    product_id: int
    quantity: int

async def validate_order(
    order: OrderCreate,
    db: Session = Depends(get_db)
) -> OrderCreate:
    product = await
db.query(Product).filter_by(id=order.product_id).first()
    if not product:
        raise HTTPException(404, "Product not found")
    if product.stock < order.quantity:
        raise HTTPException(400, f"Only {product.stock} items
available")
    return order

@app.post("/orders")
async def create_order(order: OrderCreate = Depends(validate_order)):
    # order is validated against database state
    pass
```

9. **Your API needs to support both cookie-based and header-based authentication. How?**

   Answer:

```
async def get_current_user(
    request: Request,
    db: Session = Depends(get_db)
) -> User:
    # Try header first
    auth_header = request.headers.get("Authorization")
    if auth_header and auth_header.startswith("Bearer "):
        token = auth_header.split(" ")[1]
    else:
        # Fall back to cookie
        token = request.cookies.get("access_token")

    if not token:
        raise HTTPException(401, "Not authenticated")

    user_id = decode_token(token)
    user = await db.query(User).filter_by(id=user_id).first()
    if not user:
        raise HTTPException(401, "User not found")
    return user
```

10. **How do you handle a long-running operation (> 30 seconds) in a route handler?**

    Answer: Don't do the work in the handler. Accept the request, queue the work, return immediately with a job ID.

```
@app.post("/reports", status_code=202)
async def generate_report(
    request: ReportRequest,
    background_tasks: BackgroundTasks,  # For simple cases
    # OR: task_queue = Depends(get_task_queue)  # For production
):
    job_id = str(uuid.uuid4())

    # Option 1: BackgroundTasks (simple, same process)
    background_tasks.add_task(process_report, job_id, request)

    # Option 2: Task queue (production, distributed)
    # await task_queue.enqueue(process_report, job_id, request)

    return {
        "job_id": job_id,
        "status_url": f"/reports/jobs/{job_id}"
    }

@app.get("/reports/jobs/{job_id}")
async def get_job_status(job_id: str):
    status = await get_job_status(job_id)
    return status
```

Code Analysis

11. **What's wrong with this code?**

```
db = None

@app.on_event("startup")
async def startup():
    global db
    db = await connect_db()

@app.get("/users")
async def get_users():
    return await db.query(User).all()
```

    Answer: Multiple issues: (1) Global mutable state is bad for testing and concurrency, (2) No shutdown cleanup, (3) If startup fails after db assignment, no recovery, (4) Deprecated on_event pattern. Use lifespan + app.state instead.

12. **What happens when 100 requests hit this endpoint simultaneously?**

    ```
    @app.get("/slow")
    async def slow_endpoint():
        time.sleep(1)
        return {"status": "done"}
    ```

    Answer: Disaster. `time.sleep()` is blocking and runs in the async context, so it blocks the event loop. The 100 requests are processed sequentially, taking ~100 seconds total. Fix: use `await asyncio.sleep(1)` or change to `def slow_endpoint()` so FastAPI runs it in a thread pool.

Interview Framing

When discussing FastAPI in interviews:

1. **Emphasize type-driven design**: "I like that validation, documentation, and serialization all come from type hints. It's a single source of truth that reduces bugs."
2. **Show dependency injection understanding**: "Dependencies let me write testable code. I can override any dependency in tests without mocking. The DI system handles lifecycle management automatically."
3. **Discuss async trade-offs**: "I use async for I/O-bound work with async libraries. For CPU-bound work or blocking libraries, I use sync functions so FastAPI runs them in the thread pool. Mixing blocking calls in async functions is a common mistake."
4. **Demonstrate production thinking**: "In production, I disable docs endpoints, use proper lifespan management for resource cleanup, and structure code with routers and services for maintainability."
5. **Connect to broader concepts**: "FastAPI being ASGI-based means it can handle WebSockets and long-polling efficiently. WSGI frameworks would struggle with those patterns."